

PPL Cheatsheet

Folds

$$\text{fold}_{\text{left}}(\circ, i, (e_1, \dots, e_n)) = e_n \circ (e_{n-1} \circ \dots ((e_1 \circ i)))$$
$$\text{fold}_{\text{right}}(\circ, i, (e_1, \dots, e_n)) = e_1 \circ (e_2 \circ \dots (e_n \circ i))$$

Scheme

Haskell

```
class Eq a where
    (==) :: a -> a -> Bool

    (/=) :: a -> a -> Bool
    x /= y = not (x == y)

-- We need to define only (<=)
class (Eq a) => Ord a where
    (<), (<=), (>=), (>) :: a -> a -> Bool
    max, min           :: a -> a -> a

class Show a where
    show :: a -> String

class Foldable t where
    foldr :: (a -> b -> b) -> b -> t a -> b

-- A well defined functor should obey the following rules:
-- 1. fmap id = id
-- 2. fmap (f . g) = fmap f $ fmap g
class Functor f where
    fmap :: (a -> b) -> f a -> f b

class (Functor f) => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b

-- For a monad to behave correctly, the method definitions must obey:
-- 1. return is the identity
-- 2. binds must be associative
class (Applicative m) => Monad m where
    (>=>) :: m a -> (a -> m b) -> m b

    (>>)  :: m a -> m b -> m b
```

```

m >> k = m >=> \_ -> k

return :: a -> m a
return = pure

fail :: String -> m a
fail s = error s

-- requires `import Control.Monad.State`
data State st a = State (st -> (st, a))
instance Functor (State st) where
    fmap f (State g) = State (\s -> let (s', x) = g s
                                     in (s', f x))
instance Applicative (State st) where
    pure x = State (\t -> (t, x))
    (State f) <*> (State g) =
        State (\state -> let (s, f') = f state
                          (s', x) = g s
                          in (s', f' x))
instance Monad (State state) where
    State f >=> g = State (\olds ->
        let (news, value) = f olds
        State f' = g value
        in f' news)
-- State utilities (signatures are not accurate)
runState :: State s a -> s -> (a, s) -- given a computation, run it with a given initial state
get :: State s s -- set the result value to the state and leave the state unchanged
put :: s -> State s () -- set the result value to () and set the state value
return :: a -> State s a -- set the result value but leave the state unchanged

```

Erlang

Maps

```

Map = #{one => 1, "Two" => 2, 3 => three}. % init
Map#{one := "I"}. % update/insert
#{"Two" := V} = Map. % retrieves value corresponding to "Two" and puts it into V

```

Some built-in functions

```

date().
time().
length([1,2,3,4,5]).
size({a,b,c}).
atom_to_list(an_atom). % "an_atom"
list_to_tuple([1,2,3,4]). % {1,2,3,4}
integer_to_list(2234). % "2234"
tuple_to_list({}). % []

math:sqrt(4).

rand:uniform(). % Random float f s.t. 0.0 <= f < 1.0
rand:uniform(10). % Random int n s.t. 1 <= n <= 10

lists:member(a, X). % a is in X
lists:map(fun nonLambda/2, [1,2,3]).

```

```

lists:foldr(fun nonLambda/2, 0, [1,2,3]).
lists:foldl(fun nonLambda/2, 0, [1,2,3]).
lists:sublist(List, Start, Len)
% lists:{concat,append,filter} work as usual

io:format(fmt, [Arg1, Arg2]).

apply(Mod, Func, Args). % ?MODULE is the current module, Args is the
                        % list of arguments

```

Guard language

```

number(X)          - X is a number
integer(X)         - X is an integer
float(X)           - X is a float
atom(X)            - X is an atom
tuple(X)           - X is a tuple
list(X)            - X is a list
X > Y + Z          - X is > Y + Z
X := Y             - X is exactly equal to Y
X /= Y             - X is not exactly equal to Y
X == Y             - X is equal to Y (with int coerced to floats,
                        i.e. 1 == 1.0 succeeds but 1 := 1.0 fails)
length(X) := 3     - X is a list of length 3
size(X) := 2       - X is a tuple of size 2.

```

If and case

Case does generic pattern matching, we can use any expression and match true/false to obtain a traditional if. if uses the guard sublanguage.

```

case lists:member(a, X) of
  true  -> ...;
  false -> ...
end.
if
  integer(X) -> ...;
  tuple(x)   -> ...;
  true       -> ... % like an else
end,

```

Message passing

We spawn processes with `Pid = spawn(Module, Func, Args)`. We can register a name for a process using `register(name, Pid)`. We send messages using `PidTo ! {PidFrom, Msg}`. We can receive messages using pattern matching on the sent tuples.

MWE:

```

go() ->
  Pid2 = spawn(?MODULE, loop, []),
  Pid2 ! {self(), hello},
  receive
    {Pid2, Msg} ->
      io:format("P1 ~w~n", [Msg])
  end,
  Pid2 ! stop.

```

```

loop() ->
    receive
        {From, Msg} ->
            From ! {self(), Msg},
            loop(); % functions are tail recursive
        stop -> true
    end.

```

Actions can be timed out using `receive ... after timeout -> Actions end`. A zero timeout means “check the message buffer and if empty execute the following code”. Example of an alarm

```

set_alarm(T, What) ->
    spawn(timer, set, [self(), T, What]).

```

```

set(Pid, T, Alarm) ->
    receive
        after
            T -> Pid ! Alarm
    end.

```

```

receive
    Msg -> ... ;
end

```

Let it crash

To signal that we want to handle errors we use `process_flag(trap_exit, true)` on the master process, which will spawn processes using `spawn_linked`. The message we need to handle is 'EXIT'.

MWE:

```

main(Count) ->
    register(the_master, self()), % I'm the master, now
    start_master(Count),
    unregister(the_master),
    io:format("That's all.~n").

start_master(Count) ->
    % The master needs to trap exits:
    process_flag(trap_exit, true),
    create_children(Count),
    master_loop(Count).

% This creates the linked children
create_children(0) -> ok;
create_children(N) ->
    Child = spawn_link(?MODULE, child, [0]), % spawn + link
    io:format("Child ~p created~n", [Child]),
    Child ! {add, 0},
    create_children(N-1).

master_loop(Count) ->
    receive
        {value, Child, V} ->
            io:format("child ~p has value ~p ~n", [Child, V]),
            Child ! {add, rand:uniform(10)},
            master_loop(Count);
        {'EXIT', Child, normal} ->
            io:format("child ~p has ended ~n", [Child]),

```

```

    if
        Count == 1 -> ok; % this was the last
        true -> master_loop(Count-1)
    end;
    {'EXIT', Child, _} -> % "unnormal" termination
    NewChild = spawn_link(?MODULE, child, [0]),
    io:format("child ~p has died, now replaced by ~p ~n", [Child, NewChild]),
    NewChild ! {add, rand:uniform(10)},
    master_loop(Count)
end.

child(Data) ->
    receive
        {add, V} ->
            NewData = Data+V,
            BadChance = rand:uniform(10) < 2,
            if
                BadChance ->
                    error("I'm dying"); % random error in child:
                    NewData > 30 ->
                        ok; % child ends naturally:
                    true ->
                        the_master ! {value, self(), NewData}, % there is still work to do:
                        child(NewData)
                    end
            end
    end.
end.

```